

AI Talent Intelligence Platform

Production-Grade Semantic Search, Reranking & Hybrid Scoring Pipeline for Large-Scale Candidate Ranking (100K+ Dataset)

- Hackathon Track: Artificial Intelligence & Data Science
- Author Profile: sandeepg03
- GitHub Repository: <https://github.com/sandeepg03/talent-platform>
- Core Pipeline: Bi-Encoder (FAISS) -> Cross-Encoder Reranker -> Feature Engineer -> Hybrid Scorer

Problem Statement

Bottlenecks in Modern Resume Parsing and Ranking Pipelines

- **Scale Constraints:** Processing 100K+ resumes in real-time is computationally heavy; naive $O(N^2)$ similarity checks time out.
- **Weak Keyword Matching:** Standard parser matching misses semantic context and synonyms (e.g., matching 'ML' with 'Machine Learning').
- **Lack of Transparency:** Most modern AI ranking systems function as 'black boxes' without clear, explainable reasoning for rankings.
- **Adversarial Resumes (Honeypots):** Systems are susceptible to keyword stuffing and fake profiles designed to cheat metrics.

Proposed Solution

Two-Stage Cascade Architecture with Transparent Hybrid Scoring

- Cascade Search Pipeline: High-recall bi-encoder fast retrieval followed by high-precision cross-encoder reranking.
- Explainable AI: Every rank has a deterministic, recruiter-friendly generated explanation detailing skills, experience, and signal alignment.
- Honeypot Shielding: Integrated heuristics filter out bad-faith synthetic profiles at the feature-engineering layer.
- Flexible Hybrid Scorer: Balanced combination of semantic similarity, deep cross-encoder matching, and recruiter-specific heuristics.

System Architecture

Modular End-to-End Cascade Architecture

- JD Parser: Extracts structured criteria (must-have/nice-to-have skills, years of experience, target domains).
- Embedding Stage: Encodes JD and candidate text into 384-dim dense vectors using BAAI/bge-small-en-v1.5.
- FAISS Vector Store: Performs sub-5ms IndexFlatIP inner-product search to extract top-500 candidate pool.
- Cross-Encoder Reranking: Computes deep query-resume attention on top-500 using ms-marco-MiniLM-L-6-v2, keeping top-200.
- Feature Engineering: Extracts years of experience, education level, certificate matches, open-source signals, and flags honeypots.
- Hybrid Scoring & Reasoning: Generates final weighted score (0-100) and writes detailed recruiter-centric explanation.

Technology Stack

Modern, Production-Grade open-source AI & Data Toolkit

- Embedding & Retrieval: Sentence-Transformers (BGE-Small-en-v1.5) & Facebook AI Similarity Search (FAISS).
- Deep Learning Reranking: PyTorch & Cross-Encoder Rerankers (ms-marco-MiniLM-L-6-v2).
- Schema Validation & Models: Pydantic v2 for strict type safety and structured schemas.
- Web APIs & Servers: FastAPI backend for ultra-low latency server requests & Uvicorn ASGI server.
- Frontend App: Streamlit dashboard with interactive Plotly visualization charts.
- Testing & CI/CD: Pytest framework (362 tests) & Docker + Compose + GitHub Actions CI workflows.

Dataset Overview

100,000+ Structured Candidates and Target Job Descriptions

- Scale: 100,000+ candidate profiles in JSONL format (~464MB raw text).
- Structured Profiles: Fully typed schema including experience, skills, career history, education, certifications, and GitHub signals.
- Adversarial Profiles (Honeypots): Injected synthetic profiles designed to exploit simple keyword engines.
- Target JD: Word Document (.docx) detailing structured target roles, must-have skills, and nice-to-have capabilities.

Workflow / Pipeline

Staggered Offline Precomputation vs Low-Latency Online Ranking

- Offline Phase (precompute.py):
 - Stream-parses 100k candidates from JSONL (9.5s). Truncates profiles to 96 words (matches model token limits).
 - Staggered multiprocessing pool (3 workers, 2 threads each) encodes 100k texts in ~15 minutes on CPU.
 - Persists embeddings and builds FAISS index flat-IP file to local disk artifacts.
- Online Phase (rank.py):
 - Parses job description -> retrieves top-500 via FAISS index lookup (<5ms).
 - Reranks top-500 to top-200 via cross-encoder -> Scores top-100 -> Writes explanation -> Outputs submission.csv.

Key Features

Advanced Capabilities Designed for Recruiter Workflow

- Honeypot Resilience: Automatically scores but filters out adversarial resumes using 4 strict heuristic rules.
- Recruiter-Signal Upweighting: Skills with Expert/Advanced proficiency are repeated during text building to gain higher embedding weight.
- Deterministic Explainability: Recruiter-friendly reasoning details exact skill matches, experience gap, and signal scores.
- Low-Latency API: Low-latency endpoint (/rank) computes, reranks, scores, and filters 100k candidates in seconds.
- Interactive Streamlit UI: Upload JDs, browse candidates, inspect score breakdowns, and visualize skill alignments.

AI/ML Approach

State-of-the-Art Dense Retrieval and Cross-Attention Reranking

- Bi-Encoder (bge-small-en-v1.5): Top-performing lightweight model on MTEB. Maps queries and candidates to unit-normalized space.
- FAISS Indexing: Fast inner product search (L2-norm vectors mean dot-product is equivalent to cosine similarity).
- Cross-Encoder (ms-marco-MiniLM-L-6-v2): Computes self-attention across combined query-document tokens.
- Context Truncation: Restricting candidate texts to 96 words removes tail noise, matches model limits, and speeds up CPU speed by 4x.

Scoring & Ranking Methodology

Multi-Dimensional Transparent Weighted Scoring System

- Weighted Score Formula (Normalized to [0, 100]):
 - 40% Semantic Similarity (Bi-Encoder FAISS cosine score)
 - 30% Cross-Encoder Reranker Score (Sigmoid-normalized logit)
 - 10% Years of Experience Score (Matched against JD target)
 - 10% GitHub Open Source Signal (Activity score)
 - 5% Education Alignment Score (Degree levels matched to JD)
 - 5% Certification Match Score (Exact name matching)
- Tie-Breaking: Scores rounded to 4 decimals. Ties resolved by candidate_id ascending (alphabetical order).

Scalability & Performance

CPU-Optimized Scaling and Low Footprint Memory Design

- Staggered Multiprocessing: Avoids OpenMP deadlocks on Windows CPU and eliminates cache/memory bandwidth thrashing.
- Streaming Parser: Iterator streams candidates one-by-one, keeping RAM usage under 1.5GB during 100k candidate parsing.
- Vector Space Compression: 100k vectors stored in float32 consumes only 150MB of RAM, making it perfectly runnable on local laptops.
- Cascaded Filtering: Restricting heavy neural computations to subset layers (100k -> 500 -> 200 -> 100) ensures low latency.

Challenges Faced & Mitigations

Engineering Solutions to Real-World Hardware and System Constraints

- 1. CPU Encoding Bottleneck: Running 100k long texts took 3+ hours on CPU. Resolved by pre-truncating to 96 words (4x speedup).
- 2. OpenMP Deadlocks: PyTorch multiprocessing hung on Windows. Mitigated by staggering worker startup delays by 2.0 seconds.
- 3. Memory Thrashing: Parallel workers saturated RAM. Resolved by limiting PyTorch threads inside workers to 2.
- 4. Adversarial Candidates: Fake profiles scoring high on semantic search. Mitigated by creating a 4-rule Feature Engineer checker.

Results & Accomplishments

Robust, Validated, Hackathon-Ready Platform

- Validated Submissions: Outputs `submission.csv` containing `candidate_id`, `rank` (integer), `score` (non-increasing), and `reasoning`.
- Format Compliance: Passed all validation checks of `validate_submission.py` (ties, order, ranks, shapes).
- Robust Codebase: Passed 362/362 unit and integration tests with coverage tracking.
- Containerized deployment: Fully runnable in multi-container environment via `docker-compose`.

Future Improvements

Planned Roadmap for Production Upgrades

- GPU Acceleration: Integrate automatic CUDA acceleration inside workers for sub-minute precomputations.
- Recruiter-in-the-Loop: Leverage active learning to tune scorer weights based on click/hire actions.
- Advanced Parsing: Integrate LLMs (e.g. Gemini API) to parse unstructured resumes into structured JSON profiles.
- Multi-Lingual Support: Upgrade bi-encoder to multi-lingual models to support global recruitment campaigns.

Streamlit Demo & UI Dashboard

Interactive Dashboard Providing Complete Platform Visibility

- Main Panel: Detailed table of Top-100 ranked candidates with search, filter, and CSV export capabilities.
- Score Breakdown: Transparent Plotly radar and bar charts comparing bi-encoder, cross-encoder, and heuristic scores.
- Skill Alignment: Interactive side-by-side comparison of candidate skills vs JD requirements.
- API Docs: Integrated OpenAPI/Swagger UI detailing the /rank, /health, and /top100 endpoints.

Conclusion

Production-Grade AI Talent Intelligence Platform

- AI-Powered: Integrates bi-encoders, vector stores, and cross-encoders for maximum search recall and ranking precision.
- Transparent & Fair: Provides detailed, human-readable reasonings and excludes adversarial profiles.
- High Performance: Optimized to run seamlessly on standard CPU-only hardware under memory and time constraints.
- Submission Ready: Repository pushed, validated, and documented for hackathon delivery.